

Smart-Claims Agent

Sistema Agéntico de Procesamiento Multimodal
para la Gestión de Incidencias

OBS Business School · Máster en Machine Learning e IA · Edición 2510 · 19/05/2026

1. Resumen ejecutivo

MVP de agente IA autónomo que gestiona el ciclo completo de una reclamación de siniestro para **Seguros Pepín S.A.** (empresa ficticia). El agente integra LLMs con capacidades de visión (VLM) y ejecución de herramientas (Function Calling) para analizar correos y documentos adjuntos, contrastarlos con pólizas corporativas y ejecutar automáticamente la resolución óptima (pago, rechazo o solicitud de información), reduciendo la intervención manual en back-office.

2. Arquitectura del sistema

El sistema se organiza en **5 capas funcionales** más 2 transversales:

- **Capa 1 — Canales de entrada:** Email, portal web, WhatsApp (simulado), API REST.
- **Capa 2 — Orquestación (Agente A):** LangGraph + ReAct. Gestión de estado por expediente, router de agentes y Human-in-the-Loop (HITL).
- **Capa 3 — Agentes especializados:** Agente B (validación documental), C (extracción VLM), D (cobertura RAG), E (resolución autónoma), G (fraude/OFAC).
- **Capa 4 — Datos y conocimiento:** ChromaDB para RAG de pólizas · MariaDB para log de decisiones y HITL.
- **Capa 5 — Integración simulada:** Mock APIs Python decoradas con @tool de LangChain.
- **Transversal T1 — Seguridad:** Anonimización, control de acceso, auditoría.
- **Transversal T2 — Observabilidad:** Trazas CoT visibles, métricas, dashboard Streamlit.

Flujo de una reclamación:

Cliente envía reclamación → [A] Orquestador (triaje y enrutamiento)
[G] Verificación OFAC/fraude → [B] Validación documental
[C] Extracción VLM (fotos, facturas, actas) → [D] Verificación cobertura RAG
[E] Decisión autónoma: PAGO ($\leq$ umbral) · HITL revisión humana ($>$ umbral) · RECHAZO justificado

3. Stack tecnológico

Capa / Componente	Tecnología	Estado
LLM / VLM	Claude Sonnet (claude-sonnet-4-20250514)	■ Decidido
Framework agéntico	LangGraph + LangChain	■ Decidido
RAG (pólizas)	ChromaDB 0.5.3 + LangChain	■ Decidido

Backend	Python 3.11 + FastAPI + Uvicorn	■ Operativo
ORM / async DB	SQLAlchemy 2.0 + aiomysql	■ Operativo
Base de datos	MariaDB 11.3	■ Operativo
Frontend demo	Streamlit 1.36	■ Skeleton
Contenerización	Docker + Compose (5 servicios)	■ Operativo
OCR (fallback)	Tesseract + pytesseract	■ Instalado
llama-index	Excluido del build inicial (~2 GB)	■ Sprint 3

4. Descripción de archivos del proyecto

4.1 Raíz del proyecto

Archivo	Descripción
CONTEXT_TFM.md	Documento maestro del TFM: arquitectura, cronograma, KPIs y estado actual. Knowledge document compartido con Claude.
README.md	Guía de inicio rápido del repositorio para el equipo.
docker-compose.yml	Orquesta los 5 contenedores Docker del proyecto (backend, frontend, ChromaDB, MariaDB, Adminer).
setup.sh	Script bash de inicialización: crea .env, genera __init__.py, crea directorios data/ y arranca Docker Compose.
.env / .env.example	Variables de entorno (credenciales DB, ANTHROPIC_API_KEY, umbral HITL). El .env real está en .gitignore.
.gitignore	Excluye del repositorio: .env, __pycache__, datos locales, etc.

4.2 Backend — Entrypoint

Archivo	Descripción
backend/app/main.py	Punto de entrada de FastAPI. Registra los 3 routers, configura CORS para Streamlit e inicializa la BD al arrancar.
backend/Dockerfile	Imagen Docker del backend (Python 3.11 + todas las dependencias).
backend/requirements.txt	Dependencias Python: FastAPI, LangGraph, LangChain, Anthropic SDK, SQLAlchemy, ChromaDB, Pillow/Tesseract, aiomysql.

4.3 Agentes — backend/app/agents/

Archivo	Descripción
orchestrator.py	Agente A: orquestador central. Implementa ReAct con LangGraph. Recibe una reclamación, la analiza con Claude Sonnet y enruta a agentes B/C/D/E/G según las herramientas que el LLM decide llamar. Incluye nodo HITL para casos >5.000€. Los agentes B-G son stubs pendientes.

4.4 Herramientas (Mock APIs) — backend/app/tools/

Archivo	Descripción
claim_tools.py	Las 8 herramientas @tool que el LLM puede invocar: validate_documents, extract_multimodal, check_policy, approve_payment, send_rejection, request_more_info, check_fraud, log_decision. Todas son mocks que simulan sistemas externos reales; se sustituirán por integraciones reales en Fase II.

4.5 Base de datos — backend/app/db/ y backend/db/

Archivo	Descripción
app/db/session.py	Configura la conexión async a MariaDB con SQLAlchemy + aiomysql. Lee credenciales desde variables de entorno.
app/db/models.py	Modelos ORM: Claim (expediente con estado y tipo de siniestro) y AgentDecision (log de decisiones con razonamiento CoT).
db/init.sql	Script SQL de inicialización: crea tablas claims, agent_decisions, hitl_feedback e inserta 3 expedientes de demo (CLM-001/002/003).

4.6 Routers — backend/app/routers/

Archivo	Descripción
health.py	GET /health — devuelve {status: ok, version: 0.2.0}.
claims.py	POST /api/v1/claims/ y GET /api/v1/claims/{id}. Stubs; integración con orquestrador pendiente (Entrega 2).
agents.py	GET /api/v1/agents/status — lista los 6 agentes indicando cuál está implementado (agent_a) y cuáles son stubs.

4.7 Frontend — frontend/

Archivo	Descripción
app.py	Skeleton Streamlit (una línea). El dashboard completo con CoT visible y panel HITL está pendiente de Entrega 2.
Dockerfile	Imagen Docker del frontend Streamlit.
requirements.txt	Dependencias del frontend (Streamlit y librerías auxiliares).

4.8 CI/CD — .github/workflows/

Archivo	Descripción
dependency-audit.yml	GitHub Action semanal (lunes) y en push a main. Ejecuta pip-audit y safety sobre los requirements.txt, sube reportes como artefactos y abre issues automáticos si encuentra vulnerabilidades críticas.

4.9 Scripts de auditoría (raíz, no rastreados en git)

Archivo	Descripción
dependency_audit.py	Audita dependencias Python en busca de CVEs conocidos.
comprehensive_security_audit.py	Auditoría de seguridad amplia del proyecto.
bundle_analysis.py	Analiza el tamaño del bundle de dependencias.
update_dependencies.py	Ayuda a actualizar dependencias de forma controlada.
*_report.md / *_README.md	Reportes y documentación generados por los scripts anteriores.

5. Servicios Docker

Servicio	Contenedor	Puerto	Estado
Backend FastAPI	sca-backend	:8000	■ Operativo
Frontend Streamlit	sca-frontend	:8501	■ Skeleton
ChromaDB (RAG)	sca-chromadb	:8080	■ Operativo
MariaDB	sca-mariadb	:3306	■ Operativo
Adminer (DB UI)	sca-adminer	:8082	■ Operativo

Arranque: docker compose up -d desde la raíz del repositorio.

6. Estado actual y próximos pasos

Completado:

- Infraestructura Docker operativa (5 servicios)
- Schema MariaDB: tablas claims, agent_decisions, hitl_feedback
- FastAPI operativo: /health, /api/v1/claims, /api/v1/agents/status
- Agente A — Orquestador LangGraph ReAct implementado
- Mock APIs completas (8 tools con @tool LangChain)
- Frontend Streamlit skeleton operativo
- Entrega 1 entregada (08/05/2026)

En curso — Entrega 2 (hasta 26/06/2026):

- Dataset sintético de siniestros (data/synthetic/)
 - Ingesta de pólizas en ChromaDB (scripts/ingest_policies.py)
 - Agentes B, C, D, E, G — implementación completa
 - Dashboard Streamlit completo (CoT visible, panel HITL)
 - Capítulo de Arquitectura en la memoria escrita
 - Fijar targets numéricos de los KPIs
 - Vídeo de demostración (≤ 4 min)
-

